Nexus Logistics System

Project Idea Requirements & Architectural Approval Document

1. Distinct User Roles

Branch Manager: Responsible for authenticating into branch hubs, browsing inventory datasets, monitoring stock levels against critical thresholds, and initiating restock requests.

Global Manager (M00): Responsible for receiving restock alerts across all regional branches, confirming restock requests, and communicating supply decisions back to branch hubs.

Pricing / Order / Logistics Manager: Responsible for managing dynamic pricing strategies, processing order lifecycle workflows (Normal, Bulk, Subscription), dispatching shipments, and synchronizing legacy supplier data.

2. Quality Attribute Trade-offs

Quality attributes pull in different directions, and an architect must resolve these conflicts. In the Nexus system, we balanced our quality goals as follows:

Quality Attribute Trade-off	Conflict	Resolution
Security vs Performance	Adding strict checks and login security keeps the system safe, but extra security steps can slow down system speed for users.	We use fast, simple security checks so that unauthorized users are blocked instantly without slowing down the dashboard for branch managers.
Availability vs Performance	Monitoring cloud database connections and handling network timeouts safely adds background tasks, which can affect how fast data renders on the screen.	We designed our error handling layers to detect cloud network drops quickly within **3 seconds**, while keeping normal dashboard responses under **1 second**.
Modifiability vs Testability	Writing flexible, modular code using design patterns so developers can easily add new pricing features means there are more separate parts to test.	We structured our code so developers can easily add new features in **one day**, while keeping our test checks fast and simple to run within **2 minutes**.

3. External System Integration

Cloud Database Integration (Firebase Cloud / Firestore): Direct real-time synchronization with Google Firebase Cloud Firestore via REST/SDK endpoints. The system integrates cloud-hosted document collections for real-time inventory tracking, multi-branch data synchronization, cross-hub state persistence, and automated cloud transaction handling.

4. Separated Deployable Components

Frontend Layer: Dynamic JavaScript-based web interface rendered through HTML templates, connecting asynchronously via REST APIs to manage dashboards, inventory views, and checkout forms.

Backend Layer: Server-side architecture built using Spring Boot, handling business logic, design pattern implementations, database bridges, and REST endpoints.

5. Scope Assessment

Realistic Scope for Team: The project scope is structured into four distinct, highly cohesive modules:

1. Inventory Module
2. Pricing & Promotions Module
3. Order Management Module
4. Track Shipment & Supplier Management Module